

Database Programming

COSC 304 – Introduction to Database Systems

Database Programming Overview

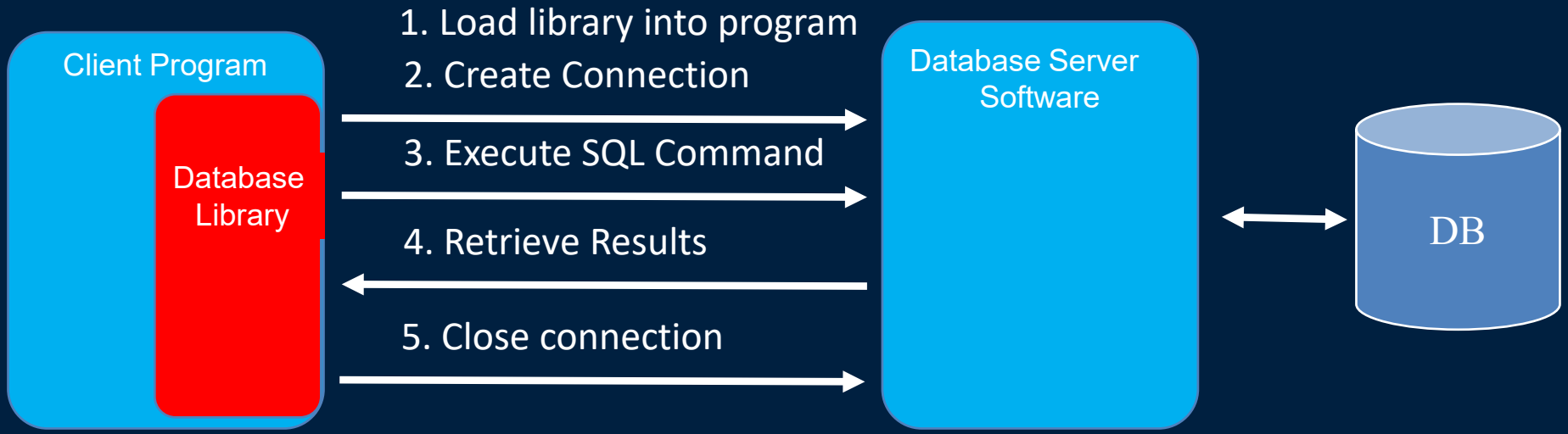
Interaction with a database is often through programs. Programming with a database requires:

- A database server (or embedded database) and its connection information
- A programming language for writing the code to query the database
- A library or driver for connecting to the particular database system

General process for programming with a database:

1) Load the database access library
2) Create a connection to the database
3) Execute a SQL command
4) Retrieve database results produced by the command
5) Close the database connection

SQL Programming Architecture



1) Load Library Error:
- Library not found in path
- Wrong library

2) Create Connection Errors:
- Invalid server URL
- Incorrect user/password
- Network issues
- Wrong library

3) Execute SQL Errors:
- Incorrect SQL
- Wrong database/table
- Improper library use

4) Retrieve Results Errors:
- Wrong column name
- Wrong column index
- Off-by-one
- Improper library use

5) Close Connection:
- FORGET TO DO IT!

SQL Programming Architecture Question

Question: **True or False:** The user has sent a SQL command to the database and an error is returned by the database server. This error is often related to the user providing an incorrect server URL.

A) true

B) false

Database Programming in Java

JDBC is the most popular method for accessing databases using Java programs.

JDBC is an ***application programming interface*** (***API***) that contains methods to connect to a database, execute queries, and retrieve results.

For each DBMS, the vendor writes a JDBC driver that implements the API. Application programs can access different DBMSs simply by changing the driver and connection information used in their program.



JDBC Interfaces

The JDBC API consists of a set of interfaces.

- A Java *interface* defines a contract specifying methods that implementing classes provide. Interfaces allow programs to use a common API without depending on a particular implementation.
- To create a JDBC driver, the DBMS vendor must implement the interfaces by defining their own classes and writing the code for the methods in the interface.

The main interfaces in the JDBC API are:

- `Driver` - The interface used by a JDBC driver to establish database connections.
- `Connection` - For connecting to the DB using the driver.
- `Statement` - For executing a query using a connection.
- `ResultSet` - For storing and manipulating results returned by a `Statement`.
- `DatabaseMetaData` - For retrieving metadata (schema) information from a database.

JDBC Program Example

```
import java.sql.*;
public class TestJDBCMySQL
{    public static void main(String[] args)
     {    String url = "jdbc:mysql://localhost/WorksOn";
          String uid = "user";
          String pw = "testpw";
          try (Connection con = DriverManager.getConnection(url, uid, pw);
               Statement stmt = con.createStatement();)
          {
               ResultSet rst = stmt.executeQuery("SELECT ename,salary FROM emp");
               System.out.println("Employee Name,Salary");
               while (rst.next())
                System.out.println(rst.getString("ename")+","+rst.getDouble("salary"));
          }
          catch (SQLException ex)
          { System.out.println(ex); }
     }
}
```

Import JDBC API

DB Connection Info

Make DB connection

Execute statement

Create statement

Iterate through ResultSet

Using try-with-resources syntax so Statement and Connection objects closed at end of try

JDBC `Driver` Interface

The `Driver` interface allows a connection to be made to a database through the driver.

Note that you do not call the `Driver` interface directly to get a connection. Drivers self-register with the `DriverManager` so `Class.forName()` is no longer needed.

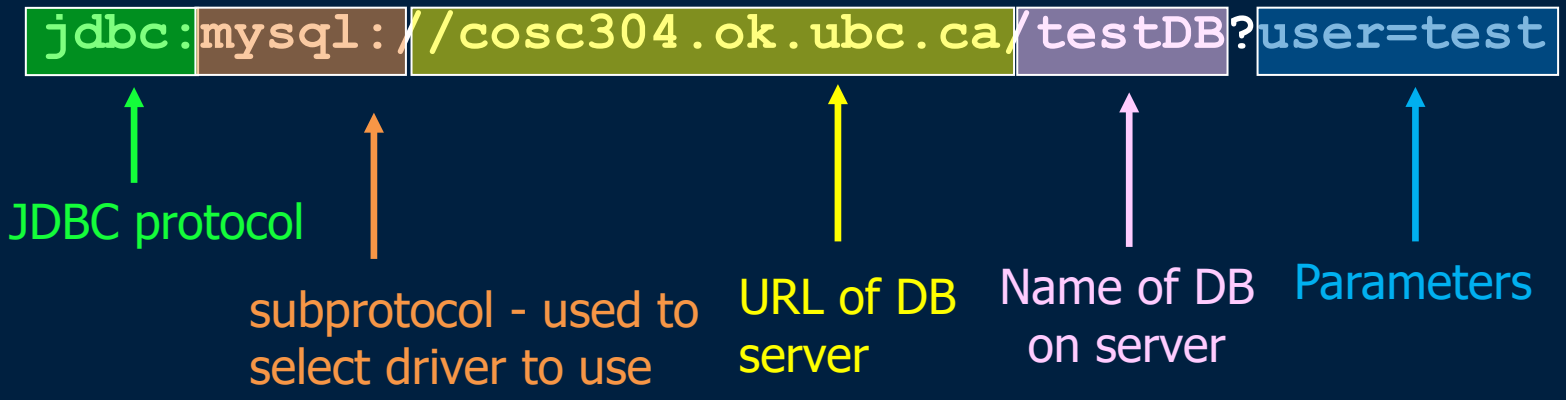
When you call `DriverManager.getConnection()`, the `DriverManager` will attempt to locate a suitable driver using the URL.

JDBC URL

JDBC URL has the typical form:

```
jdbc:subprotocol://server/database?params
```

Example:



JDBC `Connection` Interface

The `Connection` interface contains methods for managing a connection or session.

- A connection is opened after the call to `getConnection()` and should be closed when you are done.
- The `Connection` interface is used to create statements for execution on the database.

```
Connection con = DriverManager.getConnection(url, uid, pw);
Statement stmt = con.createStatement();
...
con.close();  // Auto-close with try-with-resources preferred
```

`Connection` Interface and MetaData

A `Connection` to a database can also be used to retrieve the database metadata (or schema).

- This is useful when you are writing generic tools where you do not know the schema of the database that you are querying in advance.

The method `getMetaData()` can be used to retrieve a `DatabaseMetaData` object.

DatabaseMetaData Example

```
DatabaseMetaData md = con.getMetaData();

try
(ResultSet tables= md.getTables(con.getCatalog(), null, "%", new String[]{"TABLE"}))
{
    while (tables.next())
     {
          String name = tables.getString("TABLE_NAME");
          System.out.println("Table: " + name);

          try (ResultSet cols = md.getColumns(con.getCatalog(), null, name, "%"))
          {
             while (cols.next())
               {
                  System.out.println("   " + cols.getString("COLUMN_NAME"));
             }
          }
    }
}
```

JDBC `Statement` Interface

The `Statement` interface contains methods for executing a SQL statement and returning the results it produces.

```
Statement stmt = con.createStatement();
ResultSet rst = stmt.executeQuery("SELECT ename, salary FROM emp");
```

- The `Statement` object is created by `Connection.createStatement()`.
- The statement is then executed by calling `executeQuery()` and passing the SQL string to execute.

JDBC `Statement` Interface (2)

Two common variations of executing statements are:

- 1) The `Statement` executed is an `INSERT`, `UPDATE`, or `DELETE` and no results are expected to be returned (`int` return type):

```
rowcount = stmt.executeUpdate("UPDATE emp Set salary=0");
```

- 2) The `Statement` executed is an `INSERT` which is creating a new record in a table whose primary key field is an autonumber field:

```
rowcount = stmt.executeUpdate(
                "INSERT INTO Product (name) VALUES ('Prod1')",
                Statement.RETURN_GENERATED_KEYS );
try (ResultSet keys = stmt.getGeneratedKeys())
{   if (keys.next())
    { int newId = keys.getInt(1);
    }
}
```

PreparedStatement and CallableStatement

There are two special types of `Statement` objects:

- `PreparedStatement` - extends `Statement` and is used to execute precompiled SQL statements.
  - Useful to prevent SQL injection attacks and when executing the same statement multiple times as the DBMS can optimize its parsing and execution.

```
String SQL = "UPDATE emp SET salary = ? WHERE eno = ?";
PreparedStatement pstmt = con.prepareStatement(SQL);
pstmt.setBigDecimal(1, new BigDecimal("55657.34"));
pstmt.setString(2,"E1");
int rowcount = pstmt.executeUpdate();
```

- `CallableStatement` - extends `PreparedStatement` and is used to execute stored procedures.
  - Stored procedures are precompiled SQL code stored at the database that take in parameters for their execution.

JDBC `ResultSet`

The `ResultSet` interface provides methods for manipulating the result returned by the SQL statement.

- Remember that the result is a relation (table) which contains rows and columns.
- The methods provide ways of navigating through the rows and then selecting columns of the current row.
- A `ResultSet` object maintains a cursor pointing to its current row of data. Initially the cursor is positioned before the first row.
- The `next()` method moves the cursor to the next row. It returns false when there are no more rows in the `ResultSet` object, so it can be used in a while loop to iterate through the result set.

JDBC `ResultSet` (2)

By default a `ResultSet` is not *updatable* and the cursor can move *forward-only* (can only use `next()` method).

```
while (rst.next()) {
    System.out.println(rst.getString("ename")+","+rst.getDouble(2));
}
```

- Remember, the first call to `next()` places the row cursor on the first row as the cursor starts off before the first row.
- Use the `getXXX()` methods to retrieve a particular type.
  - `getArray()`, `getBlob()`, `getBoolean()`, `getClob()`, `getDate()`, `getDouble()`, `getFloat()`, `getInt()`, `getLong()`, `getObject()`, `getString()`, `getTime()`
  - All methods take as a parameter the column index in the `ResultSet` (***indexed from 1***) or the column name and return the requested type.
  - JDBC drivers support conversions between certain SQL and Java data types. An unsupported conversion may throw a `SQLException`.

Scrollable `ResultSets`

Some drivers support `ResultSets` that allow you to navigate backwards as well as forwards.

- Request `ResultSet` type during `createStatement`.

```
// rs will be scrollable and read-only
Statement stmt = con.createStatement(
                          ResultSet.TYPE_SCROLL_INSENSITIVE,
                          ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("SELECT eno, ename FROM emp");
```

Scrollable `ResultSets` allow you to navigate in any direction through it, and these methods can now be used:

- `absolute(int row)` - set cursor to point to the given row (starting at 1)
- `afterLast()`, `beforeFirst()`, `first()`, `last()`, `next()`, `previous()`
- Scrollable ResultSets may be less efficient than forward-only ResultSets.

Updatable `ResultSets`

Updatable `ResultSets` allow you to update fields in the query result and update entire rows.

- Updatable `ResultSets` may not be supported for all queries and drivers.

Updating an existing row:

```
// rs will be scrollable and updatable record set
Statement stmt = con.createStatement(
                        ResultSet.TYPE_SCROLL_INSENSITIVE,
                        ResultSet.CONCUR_UPDATABLE);
ResultSet rs = stmt.executeQuery("SELECT eno, ename FROM emp");

rs.absolute(2);                     // Go to the 2nd row
rs.updateString(2,"Joe");           // Change name of employee
rs.updateRow();                     // Update data source
```

Updatable ResultSets (2)

Adding a new row to a ResultSet:

```
// rs will be scrollable and updatable record set
Statement stmt = con.createStatement(
                        ResultSet.TYPE_SCROLL_INSENSITIVE,
                        ResultSet.CONCUR_UPDATABLE);
ResultSet rs = stmt.executeQuery("SELECT eno, ename FROM emp");

rs.moveToInsertRow();                    // Move cursor to insert row
rs.updateString(1,"E9");
rs.updateString("ename","Joe Smith");
rs.insertRow();                          // Insert new row in DB
rs.moveToCurrentRow();                   // Move cursor to row you were on
                                         //  before insert
```

JDBC API Question

Question: Which one is not a JDBC API interface?

A) `Driver`

B) `Connection`

C) `Statement`

D) `ResultSet`

E) `HashMap`

JDBC API Question

Question: Select a **true** statement.

A) When a `ResultSet` is first opened, the cursor is positioned before the first row.

B) When a `ResultSet` is first opened, the cursor initially points to the first row with row index 1.

C) When asking for columns, the first column is index 0.

D) The method `first()` is allowed for a forward-only `ResultSet`.

Custom APIs

Many databases have custom APIs for various languages.

- For example, MySQL has APIs for C/C++ and PHP in addition to ODBC and JDBC drivers.

When to use a custom API?

- Custom APIs may be useful when the ODBC/JDBC standard does not provide sufficient functionality.
- Also useful if you *know* that the application will only access one database. (Be careful with this .. think of the future).

Custom APIs may have improved performance and increased functionality. The disadvantage is that your code is written for a specific DBMS which makes changes difficult.

- If you use a custom API, always isolate the database access code to a few general classes and methods!

Object-Relational Mapping (ORM) and Jakarta Persistence (formerly JPA)

A huge challenge with database programming is converting the database results to and from Java objects. This is called object-relational mapping, and it is tedious and error-prone.

- ***Impedance mismatch*** - Database returns values in tables and rows and Java code manipulates objects, classes, and methods.

Jakarta Persistence is a standard API and specification for mapping Java objects to relational databases. Frameworks such as Hibernate implement the specification.

JDBC Question

Write a JDBC program that connects to the `WorksOn` database on `localhost`. It lists each department (department number and name) followed by its projects (project number and name). Include departments with no projects.

Variant:

- Output in reverse order by department number. Two versions:
  - Change SQL
  - Use scrollable ResultSets (hint: previous() method).

Challenge:

- Improve your code so that it prints the department number, name, and how many projects in that department THEN the list of projects.

Conclusion

Querying databases can be done from any language with many different APIs. Querying has these steps:

1) Load the database access library
2) Create a connection to the database
3) Execute a SQL command
4) Retrieve database results produced by the command
5) Close the database connection

JDBC is an API for accessing databases using Java. Steps:

- Use a `Driver` for the database, Make a `Connection`, Create a `Statement`, Execute a query to produce a `ResultSet`, Navigate the `ResultSet` to display results or update the DB

Objectives

- Explain the common steps in querying a database using a programming language.
- Draw and explain the database-program architecture and the key components.
- List the main JDBC interfaces (`Driver`, `Connection`, `Statement`, `ResultSet`) and explain the role of each.
- Discuss the different types of `ResultSets` including scrollable and updatable `ResultSets`.
- Write a simple JDBC program (given methods of the JDBC API).
- Discuss and explain the advantages and disadvantages of using a standard API versus vendor-specific APIs.


UBC
THE UNIVERSITY OF BRITISH COLUMBIA